

Instructions:

- Answer the questions in the spaces provided on the question sheets.
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded.
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam.

Name: \_\_\_\_\_

ERP: \_\_\_\_\_

| Question: | Tracing | Sales | Design | Lists | Facebook | Inheritance | Total |
|-----------|---------|-------|--------|-------|----------|-------------|-------|
| Marks:    | 26      | 16    | 10     | 14    | 12       | 12          | 90    |
| Score:    |         |       |        |       |          |             |       |

**Question 1: Tracing** ..... 26 marks

What will be the output of the following programs? If the program has an error, write **Error** and explain the error. Each part carries 2 marks. Assume that the relevant library functions/objects are available in global namespace.

(a) `int main() {  
    [](){  
        cout << "You are " << 13 << " years old!" << endl; }; return 0; }`

**Solution:** No output. The lambda is not called.

(b) `int main() {  
    auto say_my_age = [](){  
        cout << "You are " << 13 << " years old!" << endl;  
    }; return 0; }`

**Solution:** No output. The lambda is not called.

(c) `int main(){  
    unsigned int age{13};  
    auto say_my_age = [](){  
        cout << "You are " << age << " years old!" << endl;     };  
    say_my_age(); return 0; }`

**Solution:** Compilation Error. `age` is not captured by the lambda.

(d) `int main(){  
    unsigned int age{13};  
    auto say_my_age = [age]() {  
        cout << "You are " << age << " years old!" << endl;  
    }; say_my_age(); return 0; }`

**Solution:** You are 13 years old!

```
(e) int main(){
    unsigned int age{13};
    auto say_my_age = [age](){
        cout << "You are " << age << " years old!" << endl;
        ++age;    };
    say_my_age();
    cout << "age " << age << endl;
    return 0; }
```

**Solution:** Compilation Error. **age** is captured by value and cannot be modified.

```
(f) void foo() { cout << "Hello world!" << endl; }
void bar() { cout << "Hello world!" << endl; }
int main() {
    auto ptr1 = foo;    auto ptr2 = bar;
    // Compare the function pointers
    if (ptr1 == ptr2)
        cout << "Function pointers are equal" << endl;
    else
        cout << "Function pointers are not equal" << endl;
    return 0;}
```

**Solution:** Function pointers are not equal

```
(g) template<class T>
class A {
    public:
        A() { cout << "Created\n";    }
        ~A() { cout << "Destroyed\n"; }
};
int main() { A a; return 0; }
```

**Solution:** Error in **main()**. A is a template class and requires a template argument.

```
(h) template<class T>
class A {
    public:
        A() { cout << "Created\n";    }
        ~A() { cout << "Destroyed\n"; }
};
int main()
{ A <int>a1;    A <char>a2;    A <float>a3;    return 0; }
```

**Solution:**

Created  
Created  
Created  
Destroyed  
Destroyed  
Destroyed

```
(i) template<class T>
class A {
    public:
        T func(T a, T b){ return a/b; }
};
int main() {
    A <int>a1;
    cout << a1.func(3, 2) << endl;
    cout << a1.func(3.0, 2.0) << endl; return 0; }
```

**Solution:**

1  
1 // arguments are converted to int

```
(j) template<class T, class U = char>
class A {
    T a;    U b;
    public:
        A(T a_val, char b_val = '$') {
            this->a = a_val; this->b = b_val; }
        void print() {
            cout << a << ' ' << b << endl; }
};
int main() {
    A <int, int> a1(5,10); A <int> a2(5); A <float> a3(10.0);
    return 0;
}
```

**Solution:** Compiles successfully but no output.

```
(k) template <typename T1, typename T2> // T2 is unused throughout the template
class MyClass {
    public:
        void foo(T1 t1) {
            cout << t1 << endl;
        }
        // ... all code uses only T1
};
int main() {
```

```

    MyClass<int, char> obj;
    obj.foo(42);
    return 0;
}

```

**Solution:** Compiles successfully. Output is: 42

```

(1) template <class T, int N>
class mysequence {
    T memblock [N];
public:
    void setmember(int x, T value);
    T getmember(int x);
};
template <class T, int N>
void mysequence<T,N>::setmember(int x, T value)
{ memblock[x] = value; }
template <class T, int N>
T mysequence<T,N>::getmember(int x)
{ return memblock[x]; }
int main() {
    mysequence <int, 5> myints;
    mysequence <double, 5> myfloats;
    myints.setmember (0, 100);
    myfloats.setmember (3, 3.1416);
    cout << myints.getmember(0) << '\n';
    cout << myfloats.getmember(3) << '\n';
    return 0; }

```

**Solution:**

```

100
3.1416

```

```

(m) void mystery(map<string, string> m) {
    set<string> s;
    for (auto &[k,v] : m) {
        if (k != v)
            s.insert(v);
        else
            s.erase(v);
    }
    for(auto &a : s) cout << a << " ";
    cout << "\n";
}

int main() {
    map<string,string> m1 {{"sheep","wool"}, {"house","brick"},
        {"cast","plaster"}, {"wool","wool"}};
}

```

```

map<string,string> m2 {"munchkin","blue"}, {"winkie","yellow"},
    {"corn","yellow"}, {"grass","green"}, {"emerald","green"};
map<string,string> m3 {"pumpkin","peach"}, {"corn","apple"},
    {"apple","apple"}, {"pie","fruit"}, {"peach","peach"};
mystery(m1); mystery(m2); mystery(m3);
return 0;
}

```

**Solution:**

```

brick plaster
blue green yellow
apple fruit peach

```

**Question 2: Sales** ..... 16 marks

We are given sales data as a vector of pairs. Each pair consists of a product ID (integer) and quantity sold (integer). E.g.,

```

std::vector<std::pair<int, int>> salesData = {
    {101, 5}, {102, 10}, {103, 15}, {101, 3}, {102, 7}, {103, 8}
};

```

Recall that we can access the first and second elements of a pair using **first** and **second** methods, respectively. For example, **salesData[0].first** returns **101** and **salesData[0].second** returns **5**.

- (a) [4 marks] We can use **std::map** to store total sales for each product, where the key is the product ID and the value is the total quantity sold. Write a function **map<int, int> getTotalSales(const vector<pair<int, int>> &salesData)** that returns total sales of each product as a map.

**Solution:**

```

map<int, int> getTotalSales(const vector<pair<int, int>> &salesData) {
    map<int,int> tS;
    for(auto &[a,b] : salesData)
        tS[a] += b;
    return tS;
}

```

- (b) [4 marks] Write a function **void printTotalSales(const map<int, int> &totalSales)** that prints the total sales of each product.

**Solution:**

```

void printTotalSales(const map<int, int> &totalSales) {
    for(auto &[a,b] : totalSales)
        cout << "item: " << a << " : " << "sale: " << b << "\n";
}

```

- (c) [4 marks] Using a suitable lambda expression and **std::any\_of**, write an expression to find if any product has sold more than 20 units.

**Solution:**

```
cout << std::boolalpha
      << std::any_of(totalSales.cbegin(), totalSales.cend(),
                     [](const std::pair<int,int>& p){ return p.second >= 20; } );
```

- (d) [4 marks] Using a suitable lambda expression and `std::max_element`, write an expression to find the product with the highest sales.

Hint: Call `std::max_element(totalSales.begin(), totalSales.end(), comp)`, where `comp` is a lambda expression that compares two pairs. It returns an iterator to the maximum element.

**Solution:**

```
cout << std::max_element(totalSales.cbegin(), totalSales.cend(),
                        [](const pair<int,int>& a, const pair<int,int>& b) {
                            return a.second < b.second;} ) -> second;
```

**Question 3: Design** ..... 10 marks

Define a class called **Office** that keeps track of information about an office's features. The class should have the following public methods:

| Method                                                              | Description                                                                                                                                                                                      |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Office(double width, double length, bool couch, int windows)</b> | constructs an <b>Office</b> object with the given width and length, couch (true or false) and number of windows                                                                                  |
| <b>boolean isCorner()</b>                                           | returns <b>true</b> if the office is a corner office. A corner office is square (has the same width and length) and has exactly two windows.                                                     |
| <b>string toString()</b>                                            | returns a <b>string</b> representing the office in the format: <b>width: , length: , windows:.</b> If the office has a couch <b>", has a couch"</b> should be appended to the end of the string. |

Examples:

```
Office o1 (10.3, 10.3, true, 3);
Office o2 (14.0, 6.7, false, 3);
```

A call on `o1.toString()` and `o2.toString()` would return, respectively:

```
width: 10.3, length: 10.3, windows: 3, has a couch
width: 14.0, length: 6.7, windows: 2
```

Also make **Office** objects comparable to each other by overloading the `<` operator. **Offices** that have a greater area ( $width * length$ ) should be considered "less" than other **Offices** so that they appear at the beginning of a sorted list. **Offices** that have the same area should be ordered by the number of windows they have, with **Offices** that have a greater amount of windows considered "less" than **Offices** that have less. If **Offices** still appear equal they should be compared by whether or not they have a couch. **Offices** with couches should be considered "less".

**Solution:**

```

class Office {
    double width, length;
    bool hasCouch;
    int windows;

    public:
        Office(double width, double length, bool couch, int windows)
            : width{width}, length{length}, hasCouch{couch}, windows{windows} {}

        bool isCorner() const {
            return width == length && windows == 2;
        }

        string toString() const {
            string result = "width: " + to_string(width) + ", length: " +
                to_string(length) + ", windows: " + to_string(windows);
            if (hasCouch)
                result += ", has a couch";
            return result;
        }

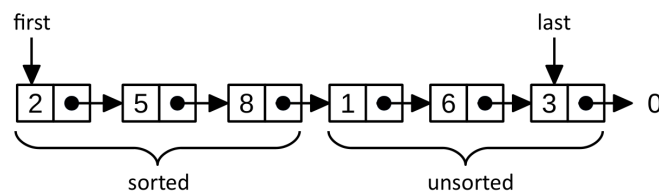
        bool operator<(const Office &other) const {
            double area = width*length, otherArea = other.width*other.length;

            if (area != otherArea)
                return area > otherArea;
            else if (windows != other.windows)
                return windows > other.windows;
            else
                return hasCouch && !other.hasCouch;
        }
};

```

**Question 4: Lists** ..... 14 marks

**struct** Node and **struct** List implement a linked list. Complete the functions below that implement the following sorting algorithm: The list is subdivided into a sorted and an unsorted part.



Initially, all list elements are in the unsorted part of the list. Iterate over each element of the list. For each element **e** of the list:

1. Find the element **x** with the smallest value, in the unsorted part of the list.

2. Swap the values of **e** and **x** using **swap**.
3. Element **e** now belongs to the sorted part of the list.

```

1  struct Node {
2      int value;
3      Node* next;
4      Node(int Value) :
5          value(Value), next(nullptr) { }
6  };
7
8  void swap(int &a, int &b) {
9      int tmp = a; a = b; b = tmp;
10 }
11
12 struct List {
13     Node* first;
14     Node* last;
15     List() :
16         first(nullptr), last(nullptr) { }
17     void append(Node* node) {
18         if (last == nullptr)
19             first = node;
20         else
21             last->next = node;
22         node->next = 0;
23         last = node;
24     }
25     Node* find_smallest(Node* from);
26     void sort();
27 };

```

- (a) [4 marks] Complete the function `List::find_smallest(Node* from)`. It must return the smallest element in the interval `[from, last]`:

```

1  Node* List::find_smallest(Node* from) {
2      Node* smallest = from;
3      Node* curr = from->next;
4      while (curr != nullptr) {
5
6          if (curr->value < smallest->value) {
7
8              smallest = curr
9          }
10
11      curr = curr -> next;
12  }
13  return smallest;
14 }

```

- (b) [4 marks] Complete the function `List::sort` that implements the sorting algorithm:

```

1  void List::sort() {
2      Node* curr = first;
3      while (curr != nullptr) {
4
5          Node* smallest = find_smallest(curr);
6
7          swap(curr->value, smallest->value);
8
9          curr = curr->next;
10     }
11 }

```

- (c) [6 marks] Add a method `removeRange` that could be added to the `List` class above. The method takes two integer parameters, `min` and `max`, and removes all elements from the list whose values are between `min` and `max`, inclusive. It should return the number of elements removed.



Suppose a `List` variable named `list` stores the following values: [4, 2, 1, 10, 15, 8, 7, 4, 20, 36, -3, 40, 5]. The call of `list.removeRange(4, 20)` would return 8 and change the `list` to store the following elements: [2, 1, 36, -3, 40].

If the value of the `max` parameter is less than that of the `min` parameter you should throw an `std::invalid_argument` exception. If the `list` is empty or does not contain any elements between or equal to `min` and `max`, it should return 0 and leave the `list` unchanged after a call to `removeRange`.

Obeey the following restrictions in your solution:

- For full credit, you should make only one pass over the list.
- Do not use auxiliary data structures such as arrays, `vector`, `queue`, `string`, etc.
- Do not modify the data field of any nodes; you must solve the problem by changing the links between nodes.

**Solution:**

```
int List::removeRange(int min, int max) {
    if (min > max)
        throw std::invalid_argument("invalid range: min greater than max");

    Node *dummy_first = new Node {0};
    dummy_first->next = first;

    int count = 0;
    Node *cur = dummy_first;
    while(cur->next) {
        if((cur->next->value >= min) && (cur->next->value <= max)) {
            Node *tbd = cur->next; // node to be deleted
            if(tbd == last) last = cur;
            cur->next = cur->next->next;
            delete tbd;
            ++count;
        }
        else
            cur = cur->next;
    }

    first = dummy_first->next;
    delete dummy_first;
    if(!first) last = nullptr;

    return count;
}
```

**Question 5: Facebook** ..... 12 marks

Create a class that represents a social network using an undirected graph.

- (a) [3 marks] Define a class `SocialNetwork` with private members to store the graph. Use `std::map` to map each user to a set of friends (`std::set`).

**Solution:**

```
class SocialNetwork {  
    map<string, set<string>> network;  
    ...  
};
```

- (b) [2 marks] Implement a method **addUser** to add a new user to the network.

**Solution:**

```
void SocialNetwork::addUser(const string& user) {  
    network[user];  
}
```

- (c) [2 marks] Implement a method **addFriendship** to create an undirected friendship between two users.

**Solution:**

```
void SocialNetwork::addFriendship(const string& user1, const string&  
    user2) {  
    network[user1].insert(user2);  
    network[user2].insert(user1);  
}
```

- (d) [2 marks] Implement a method **areFriends** to check if two users are friends.

**Solution:**

```
bool SocialNetwork::areFriends(const string& user1, const string& user2){  
    return (network.find(user1) != network.end()) &&  
        (network.find(user2) != network.end()) &&  
        (network[user1].find(user2) != network[user1].end()) &&  
        (network[user2].find(user1) != network[user2].end());  
}
```

- (e) [3 marks] Implement a method **findMutualFriends** to find mutual friends between two users.

**Solution:**

```
set<string> SocialNetwork::findMutualFriends(const string& user1, const  
    string& user2) {  
    set<string> mutualFriends;  
    if (network.find(user1) != network.end() && network.find(user2) !=  
        network.end()) {  
        for (const auto& friend1 : network[user1]) {  
            if (network[user2].count(friend1) > 0) {  
                mutualFriends.insert(friend1);  
            }  
        }  
    }  
}
```

```

        }
    }
    }
    return mutualFriends;
}

```

**Question 6: Inheritance** ..... 12 marks

```

1  class Employee {
2  protected:
3      std::string name;  int id;
4
5  public:
6      Employee(const std::string& name, int id) : name(name), id(id) {
7          std::cout << "Employee constructor called for " << name << std::endl;  }
8
9      virtual ~Employee() {  std::cout << "Employee destructor called for " << name << std::endl;  }
10
11     virtual void work() const {
12         std::cout << name << " (ID: " << id << ") is working." << std::endl;  }
13
14     std::string getName() const {  return name;  }
15
16     int getId() const {  return id;  }
17 };
18
19 class Developer : public Employee {
20 private:
21     std::string programmingLanguage;
22
23 public:
24     Developer(const std::string& name, int id, const std::string& language)
25         : Employee(name, id), programmingLanguage(language) {
26         std::cout << "Developer constructor called for " << name << std::endl;
27     }
28
29     ~Developer() {
30         std::cout << "Developer destructor called for " << name << std::endl;  }
31
32     void work() const override {
33         std::cout << name << " (ID: " << id << ") is coding in "
34             << programmingLanguage << "." << std::endl;  }
35
36     void debug() const {
37         std::cout << name << " is debugging code." << std::endl;  }
38 };
39
40 class Manager : public Employee {
41 private:
42     int teamSize;
43
44 public:
45     Manager(const std::string& name, int id, int size)
46         : Employee(name, id), teamSize(size) {
47         std::cout << "Manager constructor called for " << name << std::endl;  }
48
49     ~Manager() {
50         std::cout << "Manager destructor called for " << name << std::endl;  }
51

```

```

52     void work() const override {
53         std::cout << name << " (ID: " << id << ") is managing a team of "
54             << teamSize << " employees." << std::endl; }
55
56     void conductMeeting() const {
57         std::cout << name << " is conducting a meeting." << std::endl; }
58 };
59 int main() {
60     Employee* emp1 = new Developer("Alice", 101, "C++");
61     Employee* emp2 = new Manager("Bob", 102, 5);
62     emp1->work(); emp2->work();
63
64     Developer* dev = dynamic_cast<Developer*>(emp1);
65     if (dev) dev->debug();
66
67     Manager* mgr = dynamic_cast<Manager*>(emp2);
68     if (mgr) mgr->conductMeeting();
69
70     delete emp1; delete emp2;
71 }

```

- (a) [3 marks] List the sequence of constructor and destructor calls when the **main** function is executed. Explain the purpose and importance of having a virtual destructor in the **Employee** class.

**Solution:**

Employee constructor called for Alice  
 Developer constructor called for Alice  
 Employee constructor called for Bob  
 Manager constructor called for Bob  
 Developer destructor called for Alice  
 Employee destructor called for Alice  
 Manager destructor called for Bob  
 Employee destructor called for Bob

A virtual destructor ensures that the destructor of the derived class is called when an object is deleted through a base class pointer. This is crucial for proper resource cleanup and to avoid memory leaks. Without a virtual destructor, only the base class destructor would be called, potentially leading to incomplete destruction of derived class objects.

- (b) [3 marks] Describe what happens when the **work** method is called on **emp1** and **emp2**. Why does this behavior occur? Explain the role of the **override** keyword in the **Developer** and **Manager** classes.

**Solution:**

- When **emp1->work()** is called, it invokes the **work** method of the **Developer** class.
- When **emp2->work()** is called, it invokes the **work** method of the **Manager** class.

This behavior occurs due to polymorphism enabled by the virtual **work** method in the **Employee** class, allowing the derived class versions to be called.

The **override** keyword ensures that the **work** method in the derived classes (**Developer** and **Manager**) is intended to override a virtual method in the base class (**Employee**). It provides a compile-time check to prevent errors such as mismatched function signatures.

- (c) [3 marks] Explain the purpose of **dynamic\_cast** in the **main** function. What would happen if the

cast fails? Is it necessary to use **dynamic\_cast** for the cast from **Employee\*** to **Developer\*** and **Manager\*** in this context?

**Solution:** **dynamic\_cast** is used to safely convert a base class pointer to a derived class pointer. If the cast fails (i.e., the object is not of the target derived type), it returns **nullptr**. If the cast fails, the pointer will be **nullptr**, and the subsequent **if** check ensures that the **debug** or **conductMeeting** methods are only called if the cast was successful. In this context, **dynamic\_cast** is necessary because the types of **emp1** and **emp2** at runtime are not known at compile-time. Using **dynamic\_cast** ensures type safety and correct downcasting.

- (d) [3 marks] Why are the **name** and **id** members declared as **protected** in the **Employee** class? How does this affect the derived classes?

**Solution:** Declaring **name** and **id** as **protected** allows derived classes (**Developer** and **Manager**) to access these members directly while still restricting access from outside the class hierarchy. This provides encapsulation and allows derived classes to use these members without additional accessors.